



2

Building Our First .NET MAUI App

In this chapter, we will create a to-do list app and, in doing so, explore all the bits and pieces of what makes up an app. We will look at creating pages, adding content to pages, navigating between pages, and creating a stunning layout. Well, *stunning* might be a bit of a stretch, but we will be sure to design the app so that you can tweak it to your needs once it is complete!

The following topics will be covered in this chapter:

- Setting up the project
- Persisting data locally on a device using the repository pattern
- What MVVM is and why it's a great fit for .NET MAUI
- Using .NET MAUI pages (as views) and navigating between them using .NET MAUI controls in XAML
- Using data binding
- Using styling in .NET MAUI

Technical requirements

To complete this project, you need to have Visual Studio installed on your **Macintosh (Mac)** or PC, as well as the .NET mobile components. Refer to *Chapter 1, Introduction to .NET MAUI*, for more details on how to set up your environment. This chapter provides screenshots and instructions for Visual Studio on Windows.

This chapter will be a classic **File | New | Project** chapter, guiding you step by step through the process of creating your first to-do list app. No downloads will be required whatsoever, apart from a few NuGet packages.

You can find the full source for the code in this chapter at <https://github.com/Packt Publishing/MAUI-Projects-3rd-Edition> in the

Chapter02 folder.

Project overview

Everyone needs a way of keeping track of things. To kick-start our .NET MAUI development learning curve, we've decided that a to-do list app is the best way to get started and to help you keep track of things. A simple, classic win-win scenario.

We will start by creating a project and defining a repository to store the items of a to-do list. We will render these items in list form and allow the user to edit them using a detailed user interface. We will also look at how to store the to-do list items locally on a device through **SQLite.NET** so that they don't get lost when we exit the app.

The build time for this project is about 2 hours.

Setting up the project

.NET MAUI introduces a new code-sharing paradigm called single project. Previously, in Xamarin.Forms, you would have had a separate project for each platform your app would be deployed to. In .NET MAUI, all platforms are in a single project that is multi-targeted to all the supported platforms. By default, all code is considered shared, unless it is in one of the platform-specific subfolders. We will explore this further as we progress through this and future chapters.

Let's get started!

Creating the new project

The first step is to create a new .NET MAUI project. Open Visual Studio 2022 and select **Create a new project**:

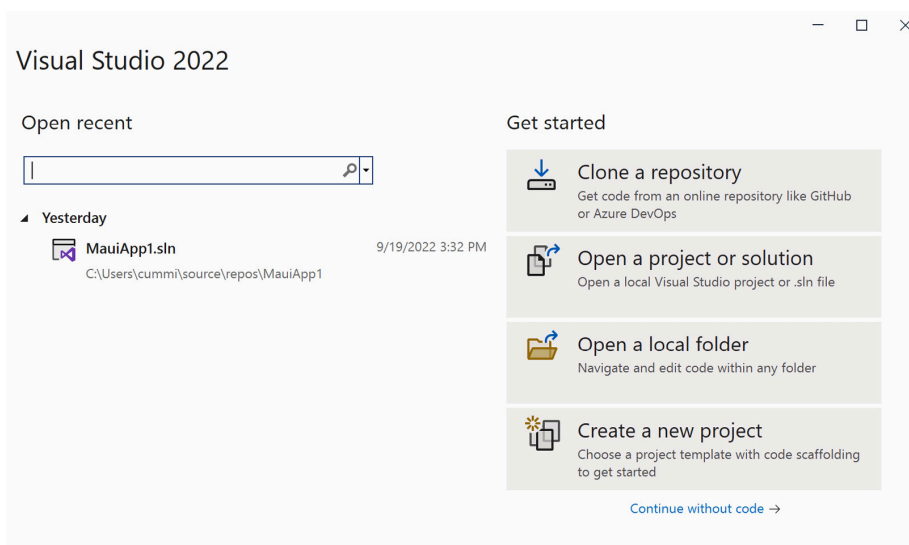


Figure 2.1 – Visual Studio 2022

This will open the **Create a new project** wizard. In the search field, type in **maui** and select the **.NET MAUI App** item from the list:

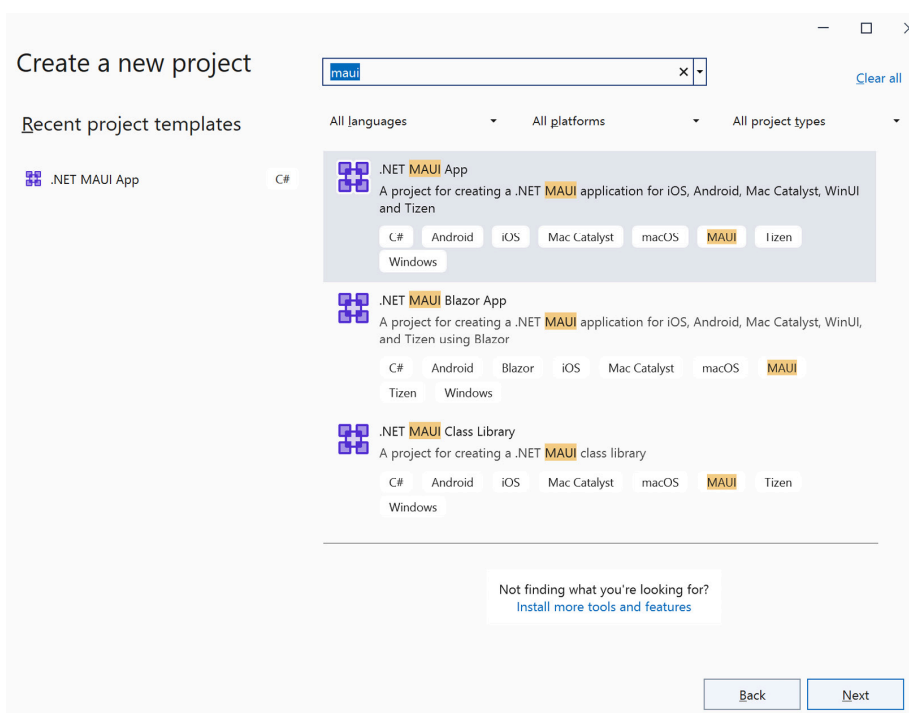
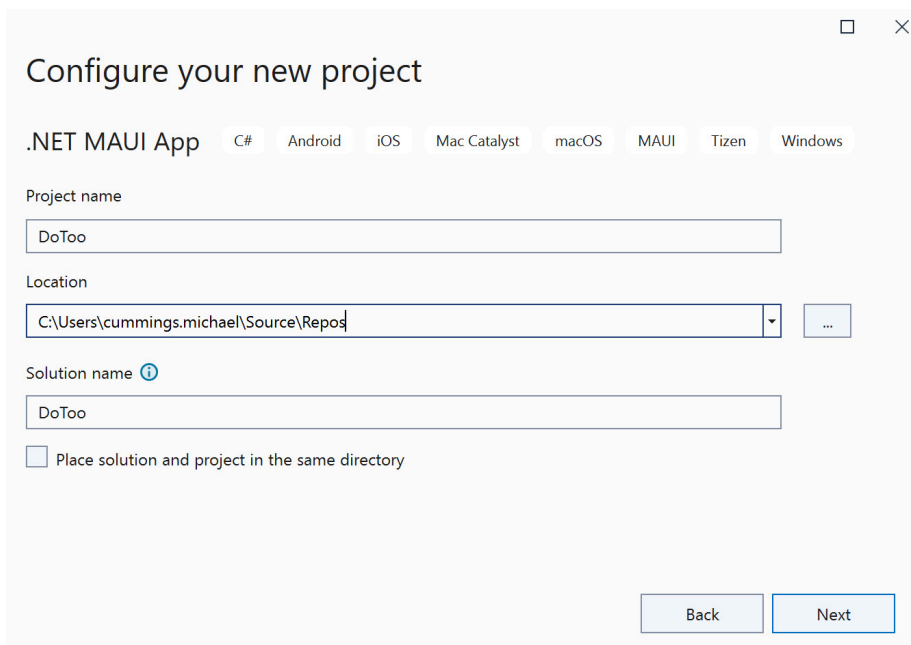


Figure 2.2 – Create a new project

Complete the next page of the wizard by naming your project **DoToo**, then click **Next**:



Configure your new project

.NET MAUI App C# Android iOS Mac Catalyst macOS MAUI Tizen Windows

Project name
DoToo

Location
C:\Users\cummings.michael\Source\Repos

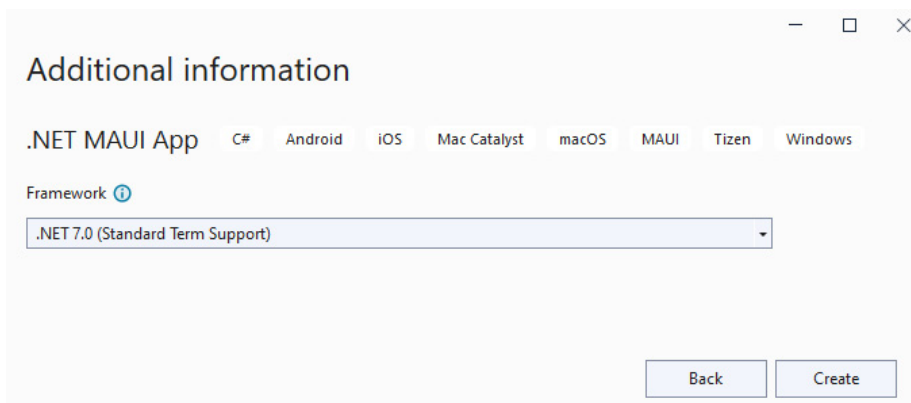
Solution name ⓘ
DoToo

☐ Place solution and project in the same directory

Back Next

Figure 2.3 – Configure your new project

The next step will prompt you for the version of .NET Core to support. At the time of writing, .NET 6 is available as **Long-Term Support (LTS)**, and .NET 7 is available as **Standard Term Support**. For this book, we will assume that you will be using .NET 7:



Additional information

.NET MAUI App C# Android iOS Mac Catalyst macOS MAUI Tizen Windows

Framework ⓘ
.NET 7.0 (Standard Term Support)

Back Create

Figure 2.4 – Additional information

Finalize the setup by clicking **Create** and wait for Visual Studio to create the project.

Congratulations! We've just created our first .NET MAUI app. Let's take a look at what the template wizard generated for us.

Examining the files

The selected template has now created a single project called **DoToo** as a .NET library that can target iOS, Mac Catalyst (macOS), Android, and

Windows platforms. You can switch the target platform using the main toolbar in Visual Studio, as shown in *Figure 2.5*:

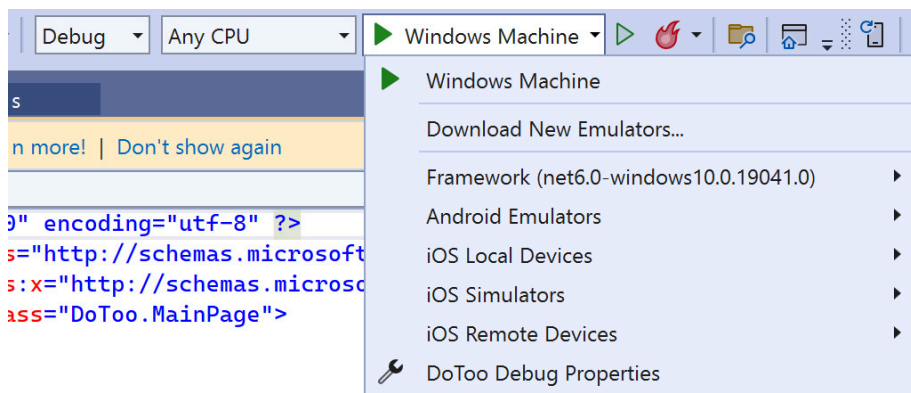


Figure 2.5 – The Debug Target drop-down menu

The Windows platform is selected by default, but you can easily switch to iOS or Android by using the **Debug Target** drop-down menu. In the drop-down under the **Framework** sub-menu, you will find all the supported target platforms.

The target framework will also change appropriately when you choose a target device. If you select an emulator under the **Android Emulators** menu item, then the Android target framework will become the current framework, whereas if you select an iOS simulator or device from one of the iOS menu items, iOS will be the current framework.

The project should now look as follows:

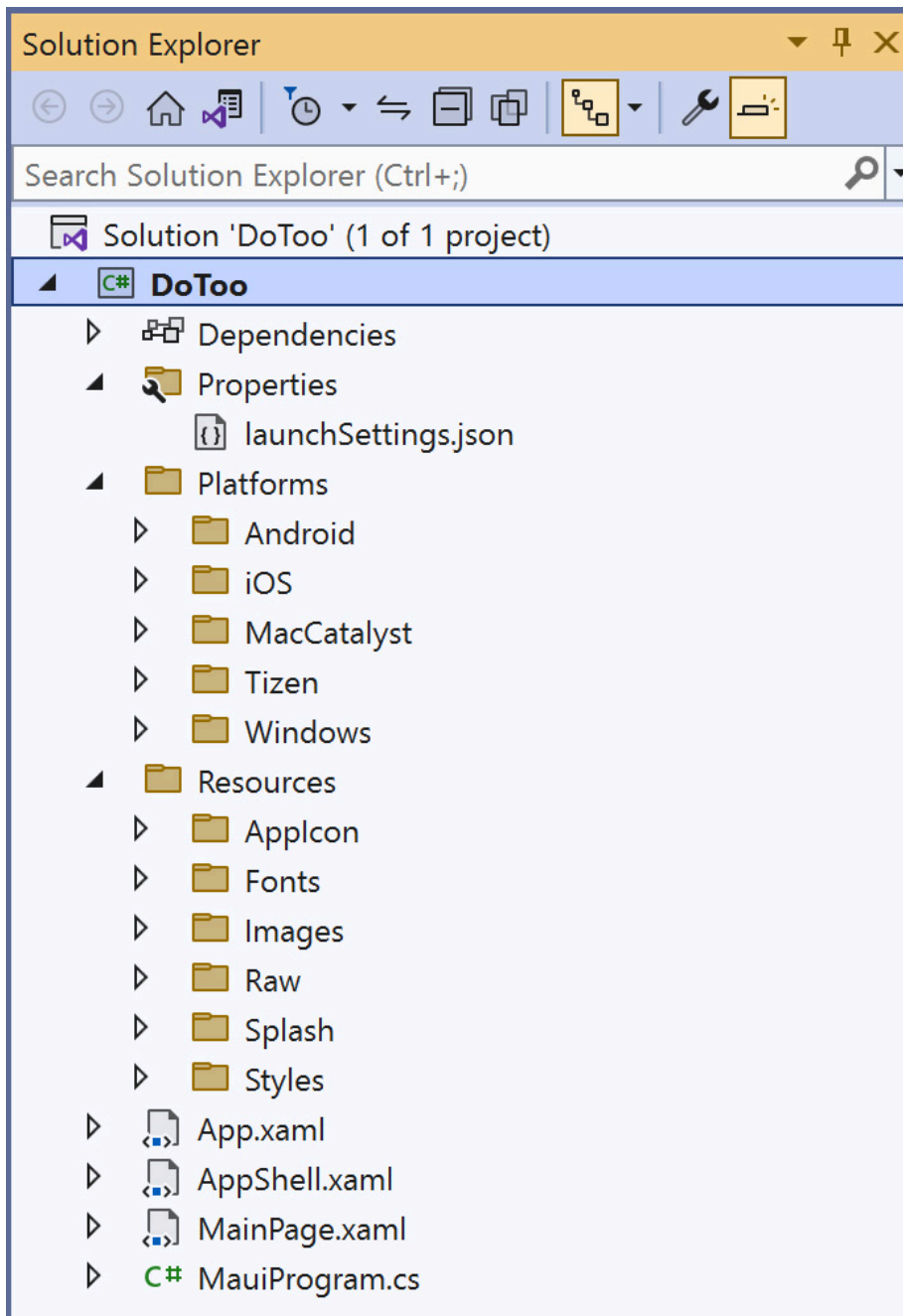


Figure 2.6 – .NET MAUI project structure

We will highlight a few important files in the project so that we have a basic understanding of what they are. First, we will look at the shared code, after which we'll look at the files/code specific to each platform (stored under the different platform folders).

Shared code

Under **Dependencies**, we will find references to any external dependencies, such as each referenced .NET mobile framework. Under each framework, you will find the .NET MAUI dependency under the **packages** folder. We will update the .NET MAUI package version in the *Updating the*

.NET MAUI packages section and add more dependencies as we progress through this chapter.

The **MauiProgram.cs** file is the starting point for the application. The initial template will generate a **MauiProgram** class that looks as follows:

```
public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf",
"OpenSansRegular");
                fonts.AddFont("OpenSans-Semibold.ttf",
"OpenSansSemibold");
            });
        #if DEBUG
            builder.Logging.AddDebug();
        #endif
        return builder.Build();
    }
}
```

The static **MauiProgram** class contains a single **CreateMauiApp** method that returns **MauiApp**. This instance is created by using **MauiAppBuilder**, which works in much the same way that the ASP.NET builders work; **MauiAppBuilder** uses a **Fluent API** to construct the **Application** instance.

WHAT IS A FLUENT API?

*A Fluent API allows method chaining wherein each method of the API returns the same context. Fluent APIs form a distinct language in themselves by using terms specific to the topic of the API. This makes the API easier to grasp and use. C#'s **Language Integrated Query (LINQ)** is a good example of a Fluent API.*

Extension methods are used to add features and services to the **MauiApp** instance. The **UseMauiApp** extension method identifies the subclass of **Microsoft.Maui.Controls.Application** to use. By default, this class is defined in the **App.xaml** and **App.xaml.cs** files. Another extension method, **ConfigureFonts**, is used by the template to register custom font files in

use by the application. Yet another example of an extension method that can be used is **ConfigureLifecycleEvents**, which is used to set up handlers for the cross-platform life cycle events available in .NET MAUI. We will discuss **ConfigureLifecycleEvents** more in *Chapter 3, Converting a Xamarin.Forms App into .NET MAUI*.

The **App.xaml** file is a XAML file that represents the app. This is a good place to put application-wide resources, which we will do later. We can also see the **App.xaml.cs** file, which contains the startup code.

If we open **App.xaml.cs**, we can see the starting point for our .NET MAUI application:

```
public partial class App : Application
{
    public App()
    {
        InitializeComponent();
        MainPage = new AppShell();
    }
}
```

The **MainPage** property is assigned to a page, which is particularly important as this determines which page is shown first to the user. In this template, this is the **DoToo.AppShell()** class.

The **AppShell.xaml** and **AppShell.xaml.cs** files declare the first visible UI component in the .NET MAUI app. Shell provides a form of navigation between pages. When you open **AppShell.xaml**, it should look like this:

```
<Shell
  x:Class="DoToo.AppShell"
  xmlns=http://schemas.microsoft.com/dotnet/2021/maui
  xmlns:x=http://schemas.microsoft.com/winfx/2009/xaml
  xmlns:local="clr-namespace:DoToo"
  Shell.FlyoutBehavior="Disabled">
  <ShellContent
    Title="Home"
    ContentTemplate="{DataTemplate local:MainPage}"
    Route="MainPage" />
</Shell>
```

The **ShellContent** element identifies an individual page that is displayed within the shell. The **ContentTemplate** attribute is used to locate the class

that implements the page – in this case, **MainPage** – while **Route** is the unique identifier for the page.

The last two files are the **MainPage.xaml** file, which contains the first page of the application, and the code-behind file, which is called **MainPage.xaml.cs**.

Next, we will work through each platform's files. Each platform has a unique folder under the **Platforms** folder. Android files are in the **Android** folder, iOS files are in the **iOS** folder, Mac Catalyst files are in the **MacCatalyst** folder, Tizen files are in the **Tizen** folder, and Windows files are in the **Windows** folder.

Android

The Android-specific platform code lives under the **Platforms/Android** folder in the project:

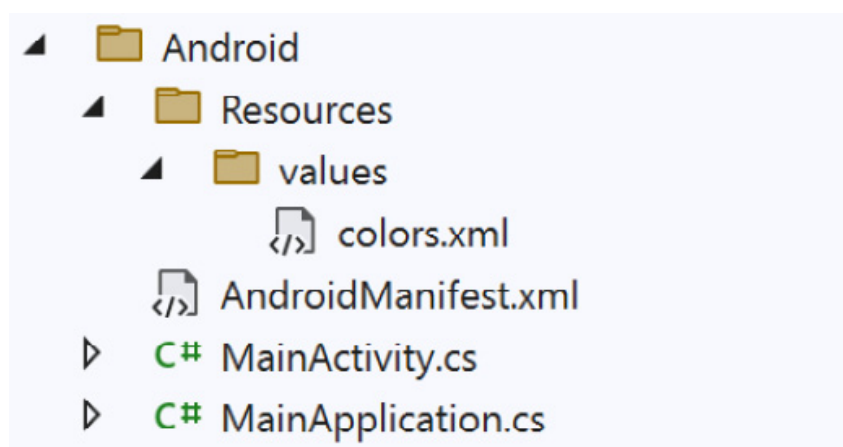


Figure 2.7 – Android-specific files

The important files here are **MainActivity.cs** and **MainApplication.cs**. These two files contain the entry point for our application when we run the app on an Android device. A standard Android app will declare **Activity** as **MainLauncher**. In .NET MAUI, this is done using the **MainLauncher** property of the **Activity** class attribute. **MauiAppCompatActivity** will search for a type decorated with **ApplicationAttribute** and instantiate it.

This attribute can be found in the **MainApplication** class in **MainApplication.cs**. During initialization, **MainApplication** will call the **CreateMauiApp** method, which, in turn, calls **MauiProgram.CreateMauiApp**, which we explored earlier in this chapter.

You don't need to understand these files in detail; just remember that they are important for initializing our app.

iOS and Mac Catalyst

The iOS and Mac Catalyst platform files are identical, but each has a folder for customizing the platform. Each platform's files are contained in their respective named folder under the **Platform** folder:

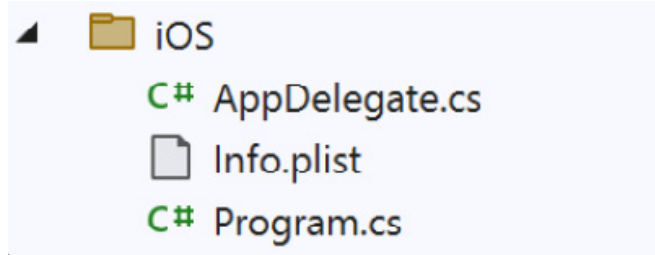


Figure 2.8 – iOS platform-specific files

AppDelegate.cs is the equivalent of the **MainApplication** class in the Android platform. It contains a single method called **CreateMauiApp** that has the same implementation as Android; it calls the **MauiProgram.CreateMauiApp** method.

The **Program.cs** file is the entry point for an iOS app. It contains the **Main** method, which calls **UIApplication.Main**, the launching point for an iOS application, and references the **AppDelegate** type to instantiate.

The code starts by initializing .NET MAUI and then loads the application. After that, it returns control to iOS. It must do this within 17 seconds; otherwise, the app is terminated by the OS.

The **info.plist** file is an iOS-specific file that contains information about the app, such as the bundle ID and its provisioning profiles. Visual Studio has a graphical editor for the **info.plist** file, but since it is a standard XML file, it can be edited in any text editor.

As with the Android app's startup code, we don't need to understand what is going on here in detail, other than that it's important for initializing our app.

Tizen

Tizen is Samsung's custom distribution of Android. The **Main.cs** file is the launching point and, like the Android platform, the **Program** class has a **CreateMauiApp** method. Tizen is not enabled by default. To enable it, fol-

low the instructions in the comments in the **DoToo.csproj** file. To develop applications for Tizen, you will need to install additional software distributed by Samsung.

Windows

The last platform we will examine is the **WinUI** app. The file structure looks as follows:

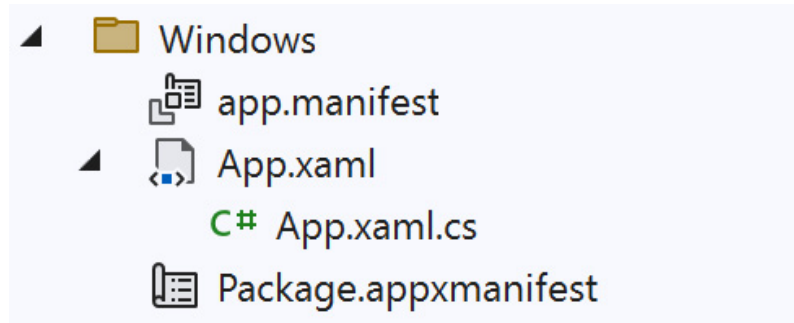


Figure 2.9 – Windows-specific files

It has an **App.xaml** file, which is like the one in the shared code, but specific to **Universal Windows Platform (UWP)** apps. It also has a related file called **App.xaml.cs**. This file is the Windows equivalent of Android's **MauiApplication**, which contains the **CreateMauiApp** method.

That's it for the platform project files. Next, we will look at how to keep .NET MAUI up to date.

Updating the .NET MAUI packages

NOTE – WINDOWS USERS

Since .NET MAUI is distributed as part of Visual Studio, it is best to allow Visual Studio to update the packages when you update Visual Studio. If you follow these steps, you can probably get .NET MAUI into an unusable state.

.NET MAUI is distributed as a set of optional **.NET workloads**. Workloads are managed from the command line using the **dotnet workload** command. To see the currently installed workloads and their versions, you can use the **dotnet workload list** command. Visual Studio 2022 has a built-in developer PowerShell to execute commands. To access it, press **Ctrl + `** on both macOS and Windows.

Running the **dotnet workload list** should give you the following output. Note that your version numbers may be higher:

```

C:\Users\cummings.michael\Source\Repos\DoToo
> dotnet workload list

```

Installed Workload Ids	Manifest Version	Installation
maui-windows	6.0.486/6.0.400	
VS 17.3.32901.215		
maui-maccatalyst	6.0.486/6.0.400	
VS 17.3.32901.215		
maccatalyst	15.4.446-ci.-release-6-0-4xx.446/6.0.400	
VS 17.3.32901.215		
maui-ios	6.0.486/6.0.400	
VS 17.3.32901.215		
ios	15.4.446-ci.-release-6-0-4xx.446/6.0.400	
VS 17.3.32901.215		
maui-android	6.0.486/6.0.400	
VS 17.3.32901.215		
android	32.0.448/6.0.400	
VS 17.3.32901.215		

Use `dotnet workload search` to find additional workloads to install.

To update the packages in the MAUI workload, you can run **dotnet workload update**. This is a sample of the result from running that command:

```

C:\Users\cummings.michael\Source\Repos\DoToo
> dotnet workload update
No workloads installed for this feature band. To update workloads
installed with earlier SDK
versions, include the --from-previous-sdk option.
Updated advertising manifest microsoft.net.sdk.android.
Updated advertising manifest microsoft.net.sdk.tvos.
Updated advertising manifest microsoft.net.sdk.macos.
Updated advertising manifest microsoft.net.sdk.maui.
Updated advertising manifest microsoft.net.workload.emscripten.
Updated advertising manifest microsoft.net.sdk.ios.
Updated advertising manifest microsoft.net.sdk.maccatalyst.
Updated advertising manifest microsoft.net.workload.mono.toolchain.
Downloading microsoft.net.sdk.android.manifest-6.0.400.msi.x64
(32.0.465)
Installing Microsoft.NET.Sdk.Android.Manifest-6.0.400.32.0.465-x64.msi
..... Done
Downloading microsoft.net.sdk.ios.manifest-6.0.400.msi.x64 (15.4.454)
Downloading microsoft.net.sdk.maccatalyst.manifest-6.0.400.msi.x64
(15.4.454)
Downloading microsoft.net.sdk.macos.manifest-6.0.400.msi.x64
(12.3.454)
Installing Microsoft.NET.Sdk.macOS.Manifest-6.0.400.12.3.454-x64.msi ..... Done

```

```

Downloading microsoft.net.sdk.maui.manifest-6.0.400.msi.x64 (6.0.540)
Installing Microsoft.NET.Sdk.Maui.Manifest-6.0.400.6.0.540-x64.msi
..... Done
Downloading microsoft.net.sdk.tvos.manifest-6.0.400.msi.x64 (15.4.454)
Downloading microsoft.net.workload.mono.toolchain.manifest-6.0.400.
msi.x64 (6.0.9)
Installing Microsoft.NET.Workload.Mono.ToolChain.Manifest-
6.0.400.6.0.9-x64.msi ..... Done
Downloading microsoft.net.workload.emscripten.manifest-6.0.400.msi.x64
(6.0.9)
Installing Microsoft.NET.Workload.Emscripten.Manifest-
6.0.400.6.0.9-x64.msi ..... Done
No workloads installed for this feature band. To update workloads
installed
with earlier SDK versions, include the --from-previous-sdk option.
Successfully updated workload(s): .

```

To see the result, just run the **dotnet workload list** command again:

```

C:\Users\cummings.michael\Source\Repos\DoToo
> dotnet workload list
Installed Workload Ids      Manifest Version
Installation Source
-----
maui-windows                6.0.540/6.0.400
  VS 17.3.32901.215
maui-maccatalyst            6.0.540/6.0.400
  VS 17.3.32901.215
maccatalyst                 15.4.446-ci.-release-6-0-4xx.446/6.0.400
  VS 17.3.32901.215
maui-ios                    6.0.540/6.0.400
  VS 17.3.32901.215
ios                         15.4.446-ci.-release-6-0-4xx.446/6.0.400
  VS 17.3.32901.215
maui-android                6.0.540/6.0.400
  VS 17.3.32901.215
android                    32.0.465/6.0.400
  VS 17.3.32901.215
Use `dotnet workload search` to find additional workloads to install.

```

Now that we have a basic understanding of how .NET MAUI projects are structured, we can start building our first app!

Creating a repository and a TodoItem model

Any good architecture always involves abstraction. In this app, we need something to store and retrieve the items of our to-do list. Later, these will be stored in a SQLite database, but adding a reference to the database directly in the code that is responsible for the GUI is generally a bad idea as it tightly couples your data storage implementation to the UI layer, making it harder to test your UI code independently from the database.

So, what we need is something to abstract our database from the GUI. For this app, we've chosen to use a simple repository pattern. This repository is simply a class that sits between the SQLite database and our upcoming **ViewModel** class. This is the class that handles the interaction with the view, which, in turn, handles the GUI.

The repository will expose methods for getting, adding, and updating items, as well as events that allow other parts of the app to react to changes in the repository. It will be hidden behind an interface so that we can replace the entire implementation later without modifying anything but a line of code in the initialization of the app. This is made possible by the **Microsoft.Extensions.DependencyInjection** NuGet package.

Defining a to-do list item

We will start by creating a **TodoItem** class, which represents a single item on the list. This is a simple **Plain Old CLR Object (POCO)** class, where **CLR** stands for **Common Language Runtime**. In other words, this is a .NET class without any dependencies on third-party assemblies. To create the class, follow these steps:

1. In the **DoToo** project, create a folder called **Models**.
2. Add a class called **TodoItem.cs** to that folder and enter the following code:

```
namespace DoToo.Models;
using System;
public class TodoItem
{
    public int Id { get; set; }
    public string Title { get; set; }
    public bool Completed { get; set; }
    public DateTime Due { get; set; }
}
```

This code is self-explanatory; it's a simple POCO class that only contains properties and no logic. We have a **Title** property that describes what we

want to be done, a flag named **Completed** that determines whether the to-do list item is completed, a **Due** date for when we expect it to be done, and a unique **Id** property that we will need later for the database.

Creating a repository and its interface

Now that we have the **ToDoItem** class, let's define an interface that describes a repository that will store our to-do list items:

1. In the **DoToo** project, create a folder called **Repositories**.
2. Create an interface called **ITodoItemRepository.cs** in the **Repositories** folder and write the following code:

```
namespace DoToo.Repositories;
using DoToo.Models;
public interface IToDoItemRepository
{
    event EventHandler<ToDoItem> OnItemAdded;
    event EventHandler<ToDoItem> OnItemUpdated;
    Task<List<ToDoItem>> GetItemsAsync();
    Task AddItemAsync(ToDoItem item);
    Task UpdateItemAsync(ToDoItem item);
    Task AddOrUpdateAsync(ToDoItem item);
}
```

WAIT, WHAT? NO DELETE METHOD?

*The eagle-eyed among you might have noticed that we are not defining a **Delete** method in this interface. This is something that should be in a real-world app. While the app that we are creating in this chapter does not support deleting items, we are quite sure that you could add this yourself if you want to!*

This interface defines everything we need for our app. It is there to create logical insulation between your implementation of a repository and the user of that repository. If any other parts of your application want an instance of **ITodoItemRepository**, we can pass it an object that implements **ITodoItemRepository**, regardless of how it's implemented.

With that said, let's implement **ITodoItemRepository**:

1. Create a class called **ToDoItemRepository.cs** in the **Repositories** folder.
2. Enter the following code:

```
namespace DoToo.Repositories;
using DoToo.Models;
public class TodoItemRepository : ITodoItemRepository
{
    public event EventHandler<TodoItem> OnItemAdded;
    public event EventHandler<TodoItem> OnItemUpdated;
    public async Task<List<TodoItem>> GetItemsAsync()
    {
        return null; // Just to make it build
    }
    public async Task AddItemAsync(TodoItem item)
    {
    }
    public async Task UpdateItemAsync(TodoItem item)
    {
    }
    public async Task AddOrUpdateAsync(TodoItem item)
    {
        if (item.Id == 0)
        {
            await AddItemAsync(item);
        }
        else
        {
            await UpdateItemAsync(item);
        }
    }
}
```

This code is the bare-bones implementation of the interface, except for the **AddOrUpdateAsync(...)** method. This handles a small piece of logic that states that if the **Id** value of an item is **0**, it's a new item. Any item with an **Id** value greater than **0** is stored in the database. This is because the database assigns a value larger than **0** when we create rows in a table.

There are also two events defined in the preceding code. They will be used to notify subscribers of a list of items that have been updated or added.

Connecting SQLite to persist data

We now have an interface, as well as a skeleton to implement that interface. The last thing we need to do to finish this section is to connect SQLite in the implementation of the repository.

Adding the SQLite NuGet package

To access SQLite in this project, we need to add a NuGet package called **sqlite-net-pcl** to the **DoToo** project. To do this, right-click on the **Dependencies** item under the **DoToo** project node of the solution and click **Manage NuGet Packages...**:

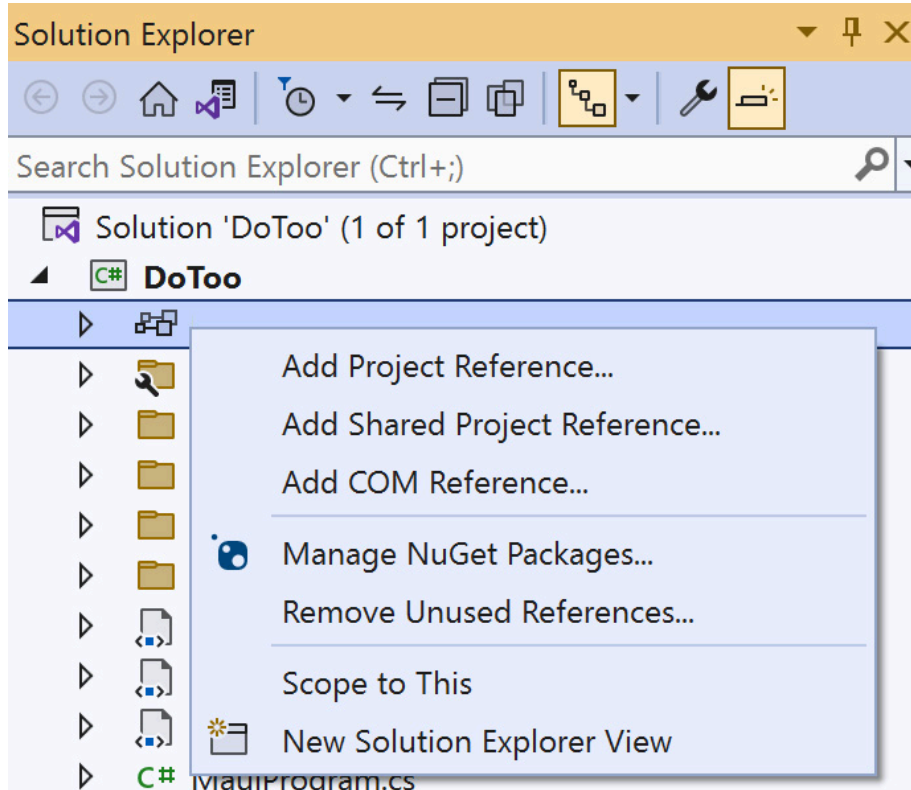


Figure 2.10 – Manage NuGet Packages...

PORTABLE CLASS LIBRARY (PCL)

*You might have noticed that the NuGet package is suffixed with **-pcl**. This is an example of what happens when naming conventions go wrong. This package supports .NET Standard 1.0, even though the name says PCL, which was the predecessor of .NET Standard.*

This brings up the **NuGet Package Manager** window:

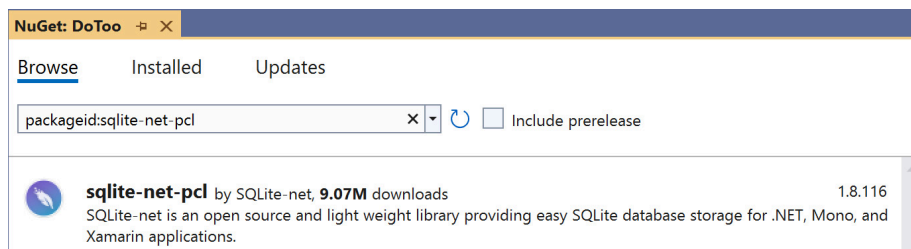


Figure 2.11 – NuGet Package Manager

To install the SQLite NuGet package, follow these steps:

1. Click **Browse** and enter **sqlite-net-pcl** in the search box.
2. Select the package by **SQLite-net** and click **Install**.
3. A dialog will be displayed showing you all the packages that will be downloaded to your system; accept the changes to complete the installation.

IMPORTANT

*Version 1.8.116 of the **sqlite-net-pcl** package references versions of the native library that are not fully compatible with .NET 6+ on all platforms. To work around this, you need to add additional references to the following packages manually with a version of at least 2.1:*

- **SQLitePCLRaw.core**
- **SQLitePCLRaw.provider.sqlite3**
- **SQLitePCLRaw.bundle_green**
- **SQLitePCLRaw.provider.dynamic_cdecl**

See <https://github.com/praeclarum/sqlite-net/issues/1102> for more details and a potential resolution.

Once the installation is complete, we can add some code to the **TodoItem** class to map the C# object to a table and create the connection to the database in the repository.

Updating the TodoItem class

Since SQLite is a relational database, it needs to know some basic information about how to create the tables that will store our objects. This is done using attributes, which are defined in the SQLite namespace:

1. Open **Models/TodoItem.cs**.
2. Add a **using SQLite** statement at the start of the file right below the **namespace** statement, as shown in the following code:

```
namespace DoToo.Models;
using SQLite;
public class TodoItem
```

3. Add the **PrimaryKey** and **AutoIncrement** attributes right before the **Id** property, as shown in the following code:

```
[PrimaryKey, AutoIncrement]
public int Id { get; set; }
```

The **PrimaryKey** attribute instructs SQLite that the **Id** property is the primary key of the table. The **AutoIncrement** attribute makes sure that the value of **Id** is increased by 1 for each new **TodoItem** class that is added to the table.

With the data object classes properly configured, it is now time to create the connection to the database.

Creating a connection to the SQLite database

We will now add all the code needed to communicate with the database. The first thing we need to do is define a connection field that will hold the connection to the database:

1. Open the **Repositories/ToDoItemRepository.cs** file.
2. Add a **using SQLite** statement right below the existing **using** statements, as shown in the following code:

```
namespace DoToo.Repositories;
using DoToo.Models;
using SQLite;
public class ToDoItemRepository : IToDoItemRepository
```

3. Add the following field right below the class declaration:

```
private SQLiteAsyncConnection connection;
```

The connection needs to be initialized. Once it is initialized, it can be reused throughout the lifespan of the repository. Since the method is asynchronous, it cannot be called from the constructor without introducing a locking strategy. To keep things simple, we will simply call it from each of the methods that are defined by the interface. To do so, add the following code to the **ToDoItemRepository** class:

```
private async Task CreateConnectionAsync()
{
    if (connection != null)
```

```

    {
        return;
    }

    var documentPath = Environment.GetFolderPath(Environment.
SpecialFolder.MyDocuments);
    var databasePath = Path.Combine(documentPath, "TodoItems.db");

    connection = new SQLiteAsyncConnection(databasePath);
    await connection.CreateTableAsync<TodoItem>();

    if (await connection.Table<TodoItem>().CountAsync() == 0)
    {
        await connection.InsertAsync(new TodoItem()
        {
            Title = "Welcome to DoToo",
            Due = DateTime.Now
        });
    }
}

```

The method begins by checking whether we already have a connection. If we do, we can simply **return**. If we don't have a connection set up, we define a path on the disk to indicate where we want the database file to be located. In this case, we will choose the **MyDocuments** folder. .NET MAUI will find the closest match to this on each platform that we target.

Then, we create the connection and store the reference to that connection in the **connection** field. We need to make sure that SQLite has created a table that mirrors the schema of the **TodoItem** table. To make developing the app easier, we add a default to-do list item if the **TodoItem** table is empty.

Next, we'll add the implementation for the database operations.

Implementing the **GetItemsAsync()**, **AddItemsAsync()**, and **UpdateItemsAsync()** methods

The only thing left to do in the repository is to implement the methods for getting, adding, and updating items:

1. Locate the **GetItemsAsync()** method in the **TodoItemRepository** class.
2. Update the **GetItemsAsync()** method with the following code:

```

public async Task<List<TodoItem>> GetItemsAsync()
{
    await CreateConnectionAsync();

```

```
        return await connection.Table<TodoItem>().ToListAsync();  
    }
```

To ensure that the connection to the database is valid, we call the **CreateConnectionAsync()** method we created in the previous section. When this method returns, we can ensure that it is initialized and that the **TodoItem** table has been created.

Then, we use the connection to access the **TodoItem** table and return a **List<TodoItem>** item that contains all the to-do list items in the database.

SQLITE AND LINQ

SQLite supports querying data using LINQ. You can play around with this after the project is complete to get a better understanding of how to work it with databases in your app.

The code for adding items is even simpler:

1. Locate the **AddItemAsync()** method in the **TodoItemRepository** class.
2. Update the **AddItemAsync()** method with the following code:

```
public async Task AddItemAsync(TodoItem item)  
{  
    await CreateConnectionAsync();  
    await connection.InsertAsync(item);  
    OnItemAdded?.Invoke(this, item);  
}
```

The call to **CreateConnectionAsync()** makes sure that we have a connection in the same way as we did for the **GetItemsAsync()** method. After this, we insert it into the database using the **InsertAsyncAsync(...)** method on the **connection** object. After an item has been inserted into the table, we invoke the **OnItemAdded** event to notify any subscribers.

The code to update an item is the same as the **AddItemAsync()** method but also includes calls to **UpdateAsync** and **OnItemUpdated**. Let's finish up by updating the **UpdateItemAsync()** method with the following code:

1. Locate the **UpdateItemAsync()** method in the **TodoItemRepository** class.
2. Update the **UpdateItemAsync()** method with the following code:

```
public async Task UpdateItemAsync(TodoItem item)
```

```
{  
    await CreateConnectionAsync();  
    await connection.UpdateAsync(item);  
    OnItemUpdated?.Invoke(this, item);  
}
```

In the next section, we'll get started with MVVM. Grab a cup of coffee and let's get started!

Using MVVM – creating views and ViewModels

Model-View-ViewModel, or **MVVM** for short, is all about **separation of concerns**. It is an architectural pattern that defines three parts, each of which has a specific meaning:

- **Model**: This relates to anything that represents data and that can be referenced with **ViewModel**.
- **View**: This is the visual component. In .NET MAUI, this is represented by a page.
- **ViewModel**: This is the class that acts as the glue between the model and the view.

We are introducing MVVM here because the MVVM pattern was designed specifically around XAML-based GUIs. This app and the rest of the apps in this book will use XAML to define the GUI and we will use the MVVM pattern to separate the code into the three aforementioned parts.

In this app, we could say that the model is the repository and the to-do list items it returns. **ViewModel** refers to this repository and exposes properties that the view can bind to. The ground rule is that any logic should reside in **ViewModel** and no logic should reside in the view. The view should know how to present data, such as converting a Boolean value into **Yes** or **No**.

MVVM can be implemented in many ways and there are quite a few frameworks that we can use to do so, such as **Prism**, **MVVMCross**, or even **TinyMvvm**. In this chapter, we have chosen to keep things simple and implement MVVM in a vanilla way first, and then use portions of the **CommunityToolkit.Mvvm** library. **CommunityToolkit.Mvvm** is an open source library produced by the .NET Foundation. It is a replacement for the **MVVMLight** library.

The main benefits of using MVVM as an architectural pattern are a clear separation of concerns, cleaner code, and great testability of **ViewModel**. If you are interested in learning more about MVVM, and how to use it with .NET MAUI, visit <https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm>.

Well, enough of that – let's write some code instead!

Defining a **ViewModel** base class

ViewModel is the mediator between the view and the model. We can benefit from it greatly by creating a common base class for all our **ViewModel** classes to inherit from. To do this, follow these steps:

1. Create a folder called **ViewModels** in the **DoToo** project.
2. Create a class called **ViewModel** in the **ViewModels** folder.
3. Add the following code:

```
using System.ComponentModel;
public abstract class ViewModel : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler PropertyChanged;
    public void RaisePropertyChanged(params string[]
propertyNames)
    {
        foreach (var propertyName in propertyNames)
        {
            PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(propertyName));
        }
    }
    public INavigation Navigation { get; set; }
}
```

The **ViewModel** class is a base class for all **ViewModel** objects. It is not meant to be instantiated on its own, so we mark it as **abstract**. It implements **INotifyPropertyChanged**, which is an interface defined in **System.ComponentModel** in the .NET base class libraries. This interface only defines one thing – the **PropertyChanged** event. Our **ViewModel** class must raise this event whenever we want the GUI to be aware of any changes to a property. This can be done manually by adding code to a setter in a property, as we did in the current implementation, or by using the **CommunityToolkit.Mvvm** library. We will talk about this in more detail in the next section.

We will also take a little shortcut here by adding an **INavigation** property to **ViewModel**. This will help us with navigation later on. This is also something that can (and should) be abstracted since we don't want **ViewModel** to be dependent on .NET MAUI to be able to reuse the **ViewModel** classes on any platform.

Introducing the CommunityToolkit.Mvvm library's ObservableObject and ObservableProperty

The traditional way of implementing a **ViewModel** class is to inherit it from a base class (such as the **ViewModel** class that we defined previously) and then add code that might look as follows:

```
public class MyTestViewModel : ViewModel
{
    private string name;
    public string Name
    {
        get { return name; }
        set
        {
            if (name != value)
            {
                name = value;
                RaisePropertyChanged(nameof(Name));
            }
        }
    }
}
```

Each property that we want to add to a **ViewModel** class yields 13 lines of code. Not too bad, you might think. However, considering that a **ViewModel** class could potentially contain 10 to 20 properties, this rapidly turns into a lot of code. We can do better than this.

In just a few simple steps, we can use the CommunityToolkit.Mvvm library to automatically inject almost all the code during the build process:

1. In the **DoToo** project, install the CommunityToolkit.Mvvm NuGet package.
2. Update the **ViewModel** class so that it looks like this:

```
using CommunityToolkit.Mvvm.ComponentModel;
```



```
[ObservableObject]
public abstract partial class ViewModel
{
    public INavigation Navigation { get; set; }
}
```

We have changed the base class of our **ViewModel** class so that it has an **ObservableObject** attribute and added the **partial** modifier. This attribute will add the base implementation of **INotifyPropertyChanged** that was previously in our **ViewModel** base class automatically during the build process.

Once our base class has been modified, we can use the **ObservableProperty** attribute to automatically generate the property implementation. The result is that the test class we had previously is reduced to a single line of code per property. This makes the code base more readable because everything happens behind the scenes:

```
public partial class MyTestViewModel : ViewModel
{
    [ObservableProperty]
    string name;
}
```

There are a few things to note about the previous example. First, the class must be marked as **partial** for the **ObservableProperty** attribute to work, just like the **ObservableObject** attribute. Second, when using the **ObservableProperty** attribute, you place it on a private field, not a property. The CommunityToolkit.Mvvm library uses **Source Generators**, a feature added in .NET 5, to generate the actual property implementation.

One of the great things about using Source Generators is that you can always view the generated source to see how things work. For example, to view the generated source for the **ViewModel** class, do the following:

1. Open the **ViewModel.cs** file.
2. Right-click the **ViewModel** type name.
3. Select **Goto Implementation**.

Normally, this would do nothing, as you are in the implementation of **ViewModel**. However, since there is additional generated code, Visual Studio will show you a list of locations that contain implementations for **ViewModel**, similar to what's shown here:

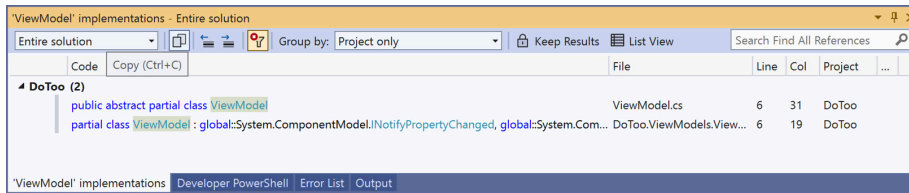


Figure 2.12 – Finding all implementations of ViewModel

The first item in the list is what we added to the **ViewModel.cs** file, and the second item in the list is the generated code. By double-clicking the item, it will open the generated code in a new code window. In the *Creating TodoItemViewModel* section, you can follow the same steps to see what is generated for the property implementations.

Now that we've seen how to implement properties using a sample **ViewModel**, it is time to create the concrete **ViewModel** classes.

Creating MainViewModel

So far, we have mainly prepared to write the code that will make up the app itself. **MainViewModel** is the **ViewModel** class for the first view that is displayed to the user. It is responsible for providing data and logic to a list of to-do list items. We will create the bare-bones **ViewModel** classes and add code to them as we progress through this chapter:

1. Create a class called **MainViewModel** in the **ViewModels** folder.
2. Add the following template code and resolve the references:

```
public class MainViewModel : ViewModel
{
    private readonly ITodoItemRepository repository;
    public MainViewModel(ITodoItemRepository repository)
    {
        this.repository = repository;
        Task.Run(async () => await LoadDataAsync());
    }
    private async Task LoadDataAsync()
    {
    }
}
```

The structure of this class is something that we will reuse for all the **ViewModel** classes to come.

Let's summarize the important features we want the **ViewModel** class to have:

- We inherit from the **ViewModel** class to gain access to shared logic, such as the **INotifyPropertyChanged** interface and common navigation code.
- All dependencies to other classes, such as repositories and services, which are passed through the constructor of **ViewModel**. This is handled by the **dependency injection** pattern and, more specifically in our case, by **Microsoft.Extensions.DependencyInjection**, which is the implementation of the dependency injection we are using. We will add support for automatic dependency injection in the *Wiring up dependency injection* section.
- We use an asynchronous call to **LoadDataAsync()** as an entry point to initialize the **ViewModel** class. Different MVVM libraries might do this in different ways, but the basic functionality is the same.

Creating TodoItemViewModel

TodoItemViewModel is the **ViewModel** class that represents each item in the to-do list on **MainView**. It does not have an entire view of its own, although it could have. Instead, it is rendered by a template in **ListView**. We will get back to this when we create the controls for **MainView**.

The important thing here is that this **ViewModel** object represents a single item, regardless of where we choose to render it.

Let's create the **TodoItemViewModel** class:

1. Create a class called **TodoItemViewModel** in the **ViewModels** folder.
2. Update the class so that it matches the following code:

```
namespace DoToo.ViewModels;
using CommunityToolkit.Mvvm.ComponentModel;
using DoToo.Models;
public partial class TodoItemViewModel : ViewModel
{
    public TodoItemViewModel(TodoItem item) => Item = item;
    public event EventHandler ItemStatusChanged;
    [ObservableProperty]
    TodoItem item;
    public string StatusText => Item.Completed ? "Reactivate" : "Completed";
}
```

As with any other **ViewModel** class, we inherit the **TodoItemViewModel** class from **ViewModel**. We conform to the pattern of injecting all the dependencies into the constructor. In this case, we pass an instance of the

TodoItem class to the constructor that the **ViewModel** object will use to expose the view.

The **ItemStatusChanged** event handler will be used later when we want to signal to the view that the state of the **TodoItem** class has changed. The **Item** property allows us to access the item that we passed in.

The **StatusText** property is used to make the status of the to-do item human-readable in the view.

Creating the ItemViewModel class

ItemViewModel represents the to-do list item in a view that can be used to create new items and edit existing ones:

1. In the **ViewModels** folder, create a class called **ItemViewModel**.
2. Add the following code:

```
namespace DoToo.ViewModels;
using DoToo.Repositories;
public class ItemViewModel : ViewModel
{
    private readonly IToDoItemRepository repository;
    public ItemViewModel(IToDoItemRepository repository)
    {
        this.repository = repository;
    }
}
```

The pattern is the same as for the previous two **ViewModel** classes:

- We use dependency injection to pass the **ToDoItemRepository** class to the **ViewModel** object
- We use inheritance from the **ViewModel** base class to add the common features defined by the base class

Creating the MainView view

Now that we are done with the **ViewModel** classes, let's create the skeleton code and the XAML required for the views. The template created a file named **MainPage.xml**. In MVVM, the convention is to use a **-View** suffix instead. We will also want to place all our views together in a subfolder, as we did with the **ViewModel** classes. Let's deal with the **MainPage.xml** file first, which is the view that will be loaded first:

1. Delete the **MainPage.xml** file from the root of the project.
2. Create a folder called **Views** in the **DoToo** project.
3. Right-click on the **Views** folder, select **Add**, and then click **New Item....**
4. Select **.NET MAUI** under the **C# Items** node on the left.
5. Select **.NET MAUI ContentPage (XAML)** and name it **MainView**.
6. Click **Add** to create the page:

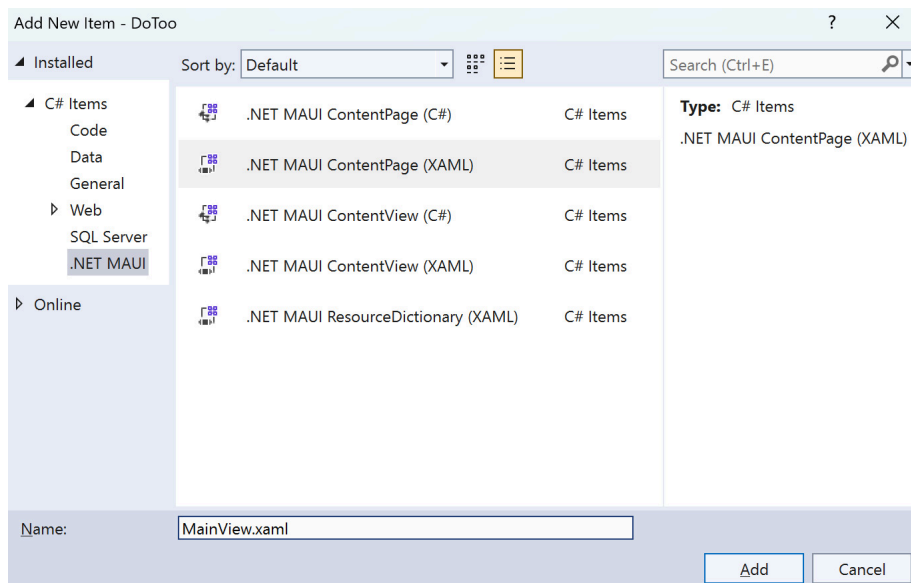


Figure 2.13 – Adding a new XAML file

Let's add some content to the newly created view:

1. Open **MainView.xaml**.
2. Remove all the template code below the **ContentPage** root node and add the XAML code highlighted in the following code:

```
<?xml version="1.0" encoding="utf-8"?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/
maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="DoToo.Views.MainView"
              Title="Do Too!">
  <ContentPage.ToolbarItems>
    <ToolbarItem Text="Add" />
  </ContentPage.ToolbarItems>
  <Grid>
    <Grid.RowDefinitions>
      <RowDefinition Height="auto" />
      <RowDefinition Height="*" />
    </Grid.RowDefinitions>
    <Button Text="Toggle filter" />
    <ListView Grid.Row="1">
    </ListView>
  </Grid>
</ContentPage>
```

```
</Grid>  
</ContentPage>
```

To be able to access custom converters, we need to add a reference to a local namespace. The `xmlns:local="clr-namespace:DoToo"` line defines this namespace for us. We will not use it directly in this case, but it's a good idea to have a local namespace defined. If we create custom controls, we can access them by writing something such as `<local:MyControl />`.

The **Title** property on the **ContentPage** page gives the page a title. Depending on the platform we are running on, the title is displayed differently. If we use a standard navigation bar, it will be displayed at the top, for example, in both iOS and Android. A page should always have a title.

The **ContentPage.ToolbarItems** node defines a toolbar item for adding new to-do items. It will also be rendered differently based on the platform, but it always follows the platform-specific UI guidelines.

A page in .NET MAUI (and in an XML document, in general) can only have one root node. The root node in a .NET MAUI page populates the **Content** property of the page itself. Since we want our **MainView** view to contain a list of items and a button at the top to toggle a filter (to switch between all items and only active items), we need to add a **Layout** control to position them on the page. **Grid** is a control that allows you to split up the available space based on rows and columns.

For our **MainView** view, we want to add two rows. The first row is a space calculated by the height of the button (**Height="auto"**) and the second row takes up all the remaining space for **ListView** (**Height="*"**). Elements such as **ListView** are positioned in the grid using the **Grid.Row** and **Grid.Column** attributes. Both properties default to **0** if they are not specified, just like the button.

ListView is a control that presents items in a list, which is coincidentally exactly what our app will do. It's worth noting that .NET MAUI does have a control called **CollectionView**, which can handle displaying collections of items better than **ListView**. Subsequent chapters will use this control, but we wanted to introduce you to the good old **ListView** control as well.

TIP

If you are interested in how **Grid** works, you can search for more information about .NET MAUI grids on the internet or check out the official documentation at <https://learn.microsoft.com/en-us/dotnet/maui/user-interface/layouts/grid>.

We also need to wire up **ViewModel** to the view. This can be done by passing the **ViewModel** class in the constructor of the view:

1. Open the code-behind file of **MainView** by expanding the **MainView.xaml** file in **Solution Explorer**. The code-behind file is named **MainView.xaml.cs**.
2. Add a **using DoToo.ViewModels** statement to the top of the file, adjacent to the existing **using** statements.
3. Modify the constructor of the class so that it looks as follows by adding the highlighted code:

```
public MainView(MainViewModel viewModel)
{
    InitializeComponent();
    viewModel.Navigation = Navigation;
    BindingContext = viewModel;
}
```

We follow the same pattern as we did with the **ViewModel** classes by passing any dependencies through the constructor. A view is always dependent on a **ViewModel** class. To simplify the project, we also assign the **Navigation** property of the page directly to the **Navigation** property defined in the **ViewModel** base class. In a larger project, we might want to abstract this property as well to make sure that we separate the **ViewModel** classes from .NET MAUI. For the sake of this app, however, it is OK to reference it directly.

Lastly, we assign **ViewModel** to the **BindingContext** class of the page. This tells the .NET MAUI binding engine to use our **ViewModel** object for the bindings that we will create later.

At this point, since we have removed **MainPage**, the project will no longer run. We will fix this in the *Making the app run* section later.

Creating the ItemView view

The second view we will add is **ItemView**. We will use this to add and edit the to-do list items:

1. Create a new content page (in the same way that we created the **MainView** view) and name it **ItemView**.
2. Edit the XAML file so that it appears as in the following code. The changes are highlighted:

```
<?xml version="1.0" encoding="UTF-8"?>
<ContentPage xmlns=http://schemas.microsoft.com/dotnet/2021/maui
              xmlns:x=http://schemas.microsoft.com/winfx/2009/xaml
              x:Class="DoToo.Views.ItemView"
              Title="New todo item">
  <ContentPage.ToolbarItems>
    <ToolbarItem Text="Save" />
  </ContentPage.ToolbarItems>
  <StackLayout Padding="14">
    <Label Text="Title" />
    <Entry />
    <Label Text="Due" />
    <DatePicker />
    <StackLayout Orientation="Horizontal">
      <Switch />
      <Label Text="Completed" />
    </StackLayout>
  </StackLayout>
</ContentPage>
```

As with **MainView**, we need a title. We will give it a default title of **New todo item** for now, but we will change this to **Edit todo item** when we reuse this view for editing later on. The user must be able to save a new or edited item, so we have added a toolbar **Save** button. The content of the page uses **StackLayout** to structure the controls. **StackLayout** adds an element vertically (the default option) or horizontally based on the space it calculates that the element takes up. This is a CPU-intensive process, so we should only use it on small portions of our layout. In **StackLayout**, we add a **Label** control, which is a line of text over the **Entry** control that comes underneath it. The **Entry** control is a text input control that contains the name of the to-do list item. Then, we have a section for **DatePicker**, where the user can select a due date for the to-do list item. The final control is a **Switch** control, which renders a toggle button to control when an item is complete, as well as a heading next to it. Since we want these to be displayed next to each other horizontally, we use a horizontal **StackLayout** control to do this.

The last step for the views is to wire up the **ItemViewModel** model to **ItemView**:

1. Open the code-behind file of **ItemView** by expanding the **ItemView.xaml** file in **Solution Explorer**.
2. Add a **using DoToo.ViewModels** statement to the top of the file, adjacent to the existing **using** statements.
3. Modify the constructor of the class so that it looks as follows. Add the code that is marked in bold:

```
public ItemView (ItemViewModel viewmodel)
{
    InitializeComponent ();
    viewmodel.Navigation = Navigation;
    BindingContext = viewmodel;
}
```

This code is identical to the code that we added for **MainView**, except for the type of **ViewModel** class.

Wiring up dependency injection

Earlier, we discussed the dependency injection pattern, which states that all dependencies, such as the repositories and view models, must be passed through the constructor of the class. This requirement has several benefits:

- It increases the readability of the code since we can quickly determine all the external dependencies
- It makes dependency injection possible
- It makes unit testing possible by mocking classes
- We can control the lifetime of an object by specifying whether it should be a singleton or a new instance for each resolution

Dependency injection is a pattern that lets us determine, at runtime, which instance of an object should be passed to a constructor when an object is created. We do this by defining a container where we register all the types of a class. We let the framework that we are using resolve any dependencies between them. Let's say that we ask the container for a **MainView** class. The container takes care of resolving **MainViewModel** and any dependencies that the class has.

.NET MAUI uses the **Microsoft.Extensions.DependencyInjection** NuGet library internally and it is exposed for us to use in our applications. The first step is to register the classes we want to participate in dependency injection.

Registering View, ViewModels, and Services

For our classes to be available through dependency injection, they need to be registered with the dependency injection service. .NET MAUI exposes the dependency injection service using the **Services** property of the **MauiAppBuilder** class. The **Services** property will return an **IServiceCollection** object, also referred to as the container.

IServiceCollection has two methods we are interested in, **AddSingleton**, and **AddTransient**. The “Transient” and “Singleton” in the method names refer to the lifetime of the objects. Transient objects are created every time they are requested from the container. Singleton objects are created only once, and that one instance is returned every time the class is requested from the container.

When registering classes with the container, it is recommended to use extension methods to group the types. For this app, there are three groups: **View**, **ViewModels**, and **Services**. The extension methods will take a single parameter and return a single value, the **MauiAppBuilder** instance. This is how the Builder pattern is implemented and allows us to chain the methods on the builder defined in the **CreateMauiApp** method.

To implement the methods, follow these steps:

1. Open the **MauiProgram.cs** file.
2. Make the following changes to the **MauiProgram** class. The changes are highlighted in bold:

```
using DoToo.Repositories;
public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf",
                    "OpenSansRegular");
                fonts.AddFont("OpenSans-Semibold.ttf",
                    "OpenSansSemibold");
            })
            .RegisterServices()
            .RegisterViewModels()
            .RegisterViews();
    }
}
```

```

        return builder.Build();
    }
    public static MauiAppBuilder RegisterServices(this
    MauiAppBuilder mauiAppBuilder)
    {
        mauiAppBuilder.Services.AddSingleton<ITodoItemRepository, TodoItemRepository>();
        return mauiAppBuilder;
    }
    public static MauiAppBuilder RegisterViewModels(this
    MauiAppBuilder mauiAppBuilder)
    {
        mauiAppBuilder.Services.AddTransient<ViewModels.
    MainViewModel>();
        mauiAppBuilder.Services.AddTransient<ViewModels.
    ItemViewModel>();
        return mauiAppBuilder;
    }
    public static MauiAppBuilder RegisterViews(this
    MauiAppBuilder mauiAppBuilder)
    {
        mauiAppBuilder.Services.AddTransient<Views.MainView>();
        mauiAppBuilder.Services.AddTransient<Views.ItemView>();
        return mauiAppBuilder;
    }
}

```

Normally, registering a type is done by using the type name as the generic argument to the registration method, as in `mauiAppBuilder.Services.AddTransient<Views.MainView>()`. But that doesn't work if you need to resolve an interface to an implementation, like what is happening in the `RegisterServices` method. There, the registration method doesn't use the generic argument; instead, it passes in the type to register as the first argument, and the second argument is the type of the instance to return.

INFO

To learn more about how `Microsoft.Extensions.DependencyInjection` works, visit <https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection> in your favorite browser.

Now that dependency injection is wired up, we can get the project running again.

Making the app run

There are just a few more changes we need to make to enable the app to run:

1. Open the **App.xaml.cs** file by expanding the **App.xaml** node in the **DoToo** project.
2. Modify the following lines in bold:

```
public App(Views.MainView view)
{
    InitializeComponent();
    MainPage = new NavigationPage(view);
}
```

3. **AppShell.Xaml** and **AppShell.xaml.cs** are no longer used, so they can be deleted from the project.

When .NET MAUI initializes the **App** class via the builder, it does so by using the dependency injection container, so any arguments you add to the **App** constructor are resolved from the container as well, and their dependencies too. In this case, we are importing the **MainView** class (and all its dependencies, including **MainViewModel** and **TodoItemRepository**) and wrapping it in **NavigationPage**. **NavigationPage** is a page defined in .NET MAUI that adds a navigation bar and enables the user to navigate to other views.

INFORMATION

*.NET MAUI includes **Shell**, and we have a whole chapter about it in this book. However, to become a good .NET MAUI developer, you need to know the basics, and the basics of navigating in .NET MAUI uses the good old **NavigationPage** control.*

That's it! Now, your project should start. Depending on the platform you are using, it might look as follows:

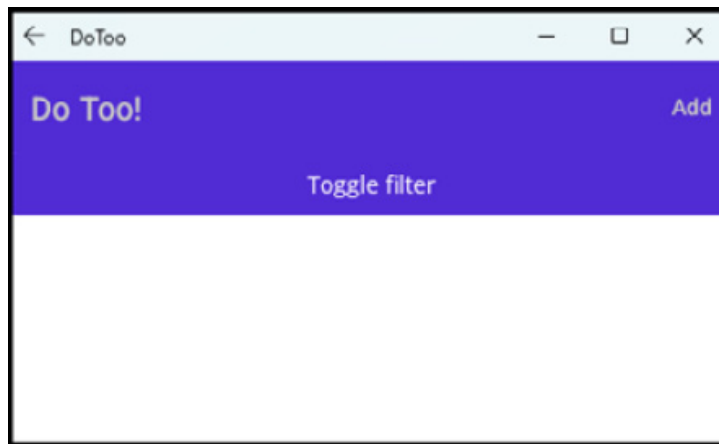


Figure 2.14 – The DoToo application in the Windows subsystem for Android

Now that we have the app running with a basic UI, let's add some functionality, starting with displaying data.

TIP

If you are debugging the app using the Windows target framework and it isn't working, and you aren't getting any error messages, try using the Android target framework. Sometimes, you can get better error reporting from a different platform.

Adding data bindings

Data binding is the heart and soul of MVVM. This is the way that the views and ViewModel communicate with each other. In .NET MAUI, we need two things to make data binding happen:

- We need an object to implement **INotifyPropertyChanged**.
- We need to set the **BindingContext** class of the page to that object. We already do this on both **ItemView** and **MainView**.

A useful feature of data binding is that it allows us to use two-way communication. For example, when data binding text to an **Entry** control, the property on the data-bound object is updated directly. Consider the following XAML:

```
<Entry Text="{Binding Title}" />
```

To make this work, we need a property named **Title** on the string object. We have to look at the documentation, define an object, and let

IntelliSense provide us with a hint to find out what type our property should be.

Controls that perform an action, such as **Button**, usually expose a property called **Command**. This property is of the **ICommand** type, and we can either return **Microsoft.Maui.Controls.Command** or an implementation of our own. The **Command** property is explained in the next section, where we will use it to navigate to **ItemView**.

In the next few sections, we will be adding the data binding and command implementations to our views and ViewModels, starting with navigating from **MainView** to **ItemView**.

INFORMATION

It's also worth noting that .NET MAUI supports one-way binding in addition to two-way data binding, which comes in handy when you want to display data in a view but not allow it to update the ViewModel. From a performance perspective, it's a good idea to mark those bindings as one-way bindings.

Navigating from MainView to ItemView to add a new item

We have an **Add** toolbar button in **MainView**. When the user taps this button, we want it to take them to **ItemView**. The MVVM way to do this is to define a command and then bind that command to the button.

In .NET MAUI, to navigate to a view, you need a reference to an instance of the destination. In this case, that would be **ItemView**. Since all our views have been registered with the dependency injection container, we will need a reference to the container to request a new instance of the view when we are ready to navigate. We will use constructor injection to have the container provide us with its instance, like this:

1. Open **ViewModels/MainViewModel.cs**.
2. Add a **using** statement for **DoToo.Views**.
3. Add the following field to the class:

```
private readonly IServiceProvider services;
```

4. Modify the constructor as follows. The changes are highlighted:

```
public MainViewModel(ITodoItemRepository repository,
    IServiceProvider services)
{
    this.repository = repository;
    this.services = services;
    Task.Run(async () => await LoadDataAsync());
}
```

This will capture the instance of **ItemView** that was created by the dependency injection container in a class field. Now, let's look at the command implementation.

All commands should be exposed as a generic **ICommand** type. This abstracts the actual command implementation, which is good general practice to follow. The command must be a property; in our case, we are creating a new **Command** object that we assign to this property. The property is read-only, which is usually fine for a **Command** object. The action of the command (the code that we want to run when the command is executed) is passed to the constructor of the **Command** object.

Following those requirements, you might end up writing something like the following:

```
public ICommand AddItem => new Command(async () =>
{
    await Navigation.PushAsync(itemView);
});
```

There is a lot of boilerplate code in that implementation that you would have to repeat for each command. This boilerplate code can get in the way of what the command is doing. Like how we were able to eliminate boilerplate code with properties, we can do the same with **ICommand**, but instead, we can use the **RelayCommand** attribute. The **RelayCommand** attribute uses a source generator to wrap a method in a new **Command** instance and expose it through a property. The property name that's generated is the method name with "Command" appended to it.

Now, we can add the **Command** object's implementation:

1. Open **ViewModels/MainViewModel.cs**.
2. Add the following method to the class:

```
[RelayCommand]
public async Task AddItemAsync() => await Navigation.
PushAsync(services.GetRequiredService<ItemView>());
```

3. Add **using CommunityToolkit.Mvvm.Input;** to the **usings** section of the file.
4. Update the class definition to allow the Source Generators to do their thing:

```
public partial class MainViewModel : ViewModel
```

The action of the command is simply to use the **Navigation** service to push the **itemView** instance onto the stack for us.

After that, we just need to wire up the **AddItemAsync** command from **ViewModel** to the **Add** button in the view:

1. Open **Views/MainView.xaml**.
2. Update the **ContentPage** element:

```
<ContentPage xmlns=http://schemas.microsoft.com/dotnet/2021/maui
              xmlns:x=http://schemas.microsoft.com/winfx/2009/xaml
              xmlns:viewModels="clr-namespace:DoToo.ViewModels"
              x:Class="DoToo.Views.MainView"
              x:DataType="viewModels:MainViewModel"
              Title="Do Too!">
```

3. Add the **Command** attribute to **ToolBarItem**:

```
<ContentPage.ToolbarItems>
  <ToolBarItem Text="Add" Command="{Binding AddItemAsyncCommand}" />
</ContentPage.ToolbarItems>
```

Run the app and tap the **Add** button to navigate to the new **ItemView** view. Notice that the back button appears automatically.

Adding new items to the list

Now that we have finished adding navigation to a new item, let's add the code to create a new item and save it to the database:

1. Open **ViewModels/ItemViewModel.cs**.
2. Add the following code in bold:


```

using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;
using DoToo.Models;
using DoToo.Repositories;
public partial class ItemViewModel : ViewModel
{
    private readonly IToDoItemRepository repository;
    [ObservableProperty]
    TodoItem item;
    public ItemViewModel(IToDoItemRepository repository)
    {
        this.repository = repository;
        Item = new TodoItem() { Due = DateTime.Now.AddDays(1) };
    }
    [RelayCommand]
    public async Task SaveAsync()
    {
        await repository.AddOrUpdateAsync(Item);
        await Navigation.PopAsync();
    }
}

```

The **Item** property holds a reference to the current item that we want to add or edit. A new item is created in the constructor and when we want to edit an item, we can simply assign our own item to this property. The new item is not added to the database unless we execute the **Save** command defined at the end. Once the item has been added or updated, we remove the view from the navigation stack and return to **MainView** again.

INFORMATION

*Since the navigation keeps pages in a stack, the framework declares methods that reflect operations that you can perform on a stack. The operation of removing the topmost item in a stack is known as popping the stack, so instead of **RemoveAsync()**, we have **PopAsync()**. To add a page to the navigation stack, we push it, so the method is called **PushAsync()**.*

Now that we have extended **ItemViewModel** with the necessary commands and properties, it's time to data-bind them in the XAML:

1. Open **Views/ItemView.xaml**.
2. Add the code marked in bold:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```

<ContentPage
  xmlns=http://schemas.microsoft.com/dotnet/2021/maui
  xmlns:x=http://schemas.microsoft.com/winfx/2009/xaml
  xmlns:viewModels="clr-namespace:DoToo.ViewModels"
  x:Class="DoToo.Views.ItemView"
  x:DataType="viewModels:ItemViewModel" >
  <ContentPage.ToolbarItems>
    <ToolbarItem Text="Save" Command="{Binding
SaveAsyncCommand}" />
  </ContentPage.ToolbarItems>
  <StackLayout Padding="14">
    <Label Text="Title" />
    <Entry Text="{Binding Item.Title}" />
    <Label Text="Due" />
    <DatePicker Date="{Binding Item.Due}" />
    <StackLayout Orientation="Horizontal">
      <Switch IsToggled="{Binding Item.Completed}" />
      <Label Text="Completed" />
    </StackLayout>
  </StackLayout>
</ContentPage>

```

The binding to the **ToolbarItems** command attribute triggers the **SaveAsync** command exposed by **ItemViewModel** when a user taps the **Save** link. It's worth noting again that any attribute called **Command** indicates that an action will take place and we must bind it to an instance of an object implementing the **ICommand** interface.

The **Entry** control that represents the title is data-bound to the **Item.Title** property of **ItemViewModel**, and the **DatePicker** and **Switch** controls bind similarly to their respective properties.

We could have exposed **Title**, **Due**, and **Complete** as properties directly on **ItemViewModel**, but instead, we chose to reuse the already-existing **TodoItem** object as a reference. This is fine, so long as the properties of the **TodoItem** object implement the **INotifyPropertyChanged** interface.

Binding ListView in MainView

A to-do list is not much use without a list of items. Let's extend **MainViewModel** with a list of items:

1. Open **ViewModels/MainViewModel.cs**.
2. Add **using System.Collections.ObjectModel** in the **using** section of the class.
3. Add a property for the to-do list items:

```
[ObservableProperty]
ObservableCollection<TodoItemViewModel> items;
```

ObservableCollection is like an ordinary collection, but it has a useful superpower: it can notify listeners about changes in the list, such as when items are added or deleted. The **ListView** control listens to changes in the list and updates itself automatically based on these. However, it's important to be aware that a change to an item in the list will not trigger an update. Changing the title of an item will not cause the list to re-render. Let's move on to implementing the rest of **MainViewModel**.

Now, we need some data:

1. Open **ViewModels/MainViewModel.cs**.
2. Replace (or complete) the **LoadDataAsync** method and create the **CreateTodoItemViewModel** and **ItemStatusChanged** methods:

```
private async Task LoadDataAsync()
{
    var items = await repository.GetItemsAsync();
    var itemViewModels = items.Select(i => CreateTodoItemViewModel(i));
    Items = new ObservableCollection<TodoItemViewModel>
(itemViewModels);
}
private TodoItemViewModel CreateTodoItemViewModel(TodoItem item)
{
    var itemViewModel = new TodoItemViewModel(item);
    itemViewModel.ItemStatusChanged += ItemStatusChanged;
    return itemViewModel;
}
private void ItemStatusChanged(object sender, EventArgs e)
{
}
```

3. Resolve all new references by adding the following **using** statements:

```
using CommunityToolkit.Mvvm.ComponentModel;
using DoToo.Models;
```

The **LoadData** method calls the repository to fetch all items. Then, we wrap each to-do list item in **TodoItemViewModel**. This contains more information that is specific to the view that we don't want to add to the **TodoItem** class. It is good practice to wrap plain objects in **ViewModel**; this makes it simpler to add actions or extra properties to it.

ItemStatusChanged is a stub that is called when we change the status of the to-do list item from **active** to **completed**, and vice versa.

We also need to hook up some events from the repository to know when data changes:

1. Open **ViewModels/MainViewModel.cs**.
2. Add the following code in bold:

```
public MainViewModel(TodoItemRepository repository,
IServiceProvider services)
{
    repository.OnItemAdded += (sender, item) =>
        items.Add(CreateTodoItemViewModel(item));
    repository.OnItemUpdated += (sender, item) =>
        Task.Run(async () => await LoadDataAsync());
    this.repository = repository;
    this.services = services;
    Task.Run(async () => await LoadDataAsync());
}
```

When an item is added to the repository, no matter who added it, **MainView** will add it to the **items** list. Since the items collection is an observable collection, the list updates. If an item is updated, we simply reload the list.

Let's data-bind our items to **ListView**:

1. Open **MainView.xaml** and locate the **ListView** element.
2. Modify it so that it reflects the following code:

```
<ListView Grid.Row="1"
    RowHeight="70" ItemsSource="{Binding Items}">
    <ListView.ItemTemplate>
        <DataTemplate x:DataType="viewModels:TodoItemViewModel">
            <ViewCell>
                <Grid Padding="15,10">
                    <Grid.RowDefinitions>
                        <RowDefinition />
                        <RowDefinition />
                    </Grid.RowDefinitions>
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="10" />
                        <ColumnDefinition Width="*" />
                    </Grid.ColumnDefinitions>
                    <BoxView Grid.RowSpan="2" />
```

```

<Label Grid.Column="1"
      Text="{Binding Item.Title}" FontSize="Medium" />
<Label Grid.Column="1" Grid.Row="1"
      Text="{Binding Item.Due}" FontSize="Micro" />
<Label Grid.Column="1" Grid.Row="1"
      HorizontalTextAlignment="End" Text="Completed"
      IsVisible="{Binding Item.Completed}"
      FontSize="Micro" />
</Grid>
</ViewCell>
</DataTemplate>
</ListView.ItemTemplate>
</ListView>

```

The **ItemsSource** binding tells **ListView** where to find the collection to iterate over and is local to **ViewModel1**. Any bindings in the **ViewCell** node, however, are local to each item that we iterate in the list. In this case, we are binding to **ToDoItemViewModel**, which contains a property named **Item**. This, in turn, has properties such as **Title**, **Due**, and **Completed**. We can navigate down the hierarchy of objects without any problem when defining a binding.

The **DataTemplate** element defines what each row will look like. We use a grid to partition the space, just as we did earlier.

You may have noticed that we didn't discuss what **BoxView** was for, and it isn't bound to any properties of **ViewModel1**. The next two sections will cover how we can use the **Completed** property to color code our items with **BoxView**.

Creating a **ValueConverter** object for the item's status

Sometimes, we want to bind to objects that are a representation of the original value. This could be a piece of text that is based on a Boolean value. Instead of **true** and **false**, for example, we might want to write **Yes** and **No** or return a color. This is where **ValueConverter** comes in handy. It can be used to convert a value to and from another value. We are going to write a **ValueConverter** object that converts the status of a to-do list item into a color:

1. In the root of the **DoToo** project, create a folder called **Converters**.
2. Create a class called **StatusColorConverter.cs** in the **Converters** folder and add the following code:

```

using System;
using System.Globalization;
public class StatusColorConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object
parameter, CultureInfo culture)
    {
        return (Color)Application.Current.Resources[
            (bool)value ? "CompletedColor" : "ActiveColor"];
    }
    public object ConvertBack(object value, Type targetType,
object parameter, CultureInfo culture)
    {
        return null;
    }
}

```

A **ValueConverter** object is a class that implements **IValueConverter**. This, in turn, only has two methods defined. The **Convert** method is called when the view reads data from **ViewModel** and the **ConvertBack** method is used when **ViewModel** gets data from the view. The **ConvertBack** method is only used for controls that return data from plain text, such as the **Entry** control.

If we look at the implementation of the **Convert** method, we'll notice that any value passed to the method is of the **object** type. This is because we don't know what type the user has bound to the property to which we are adding this **ValueConverter** class. We may also notice that we fetch colors from a resource file. We could have defined the colors in the code, but this is not recommended. So, instead, we went the extra mile and added them as a global resource to the **App.xaml** file. Resources are a good thing to take another look at once you have finished this chapter:

1. Open **App.xaml** in the **DoToo** project.
2. Add the following **ResourceDictionary** element:

```

<Application ...>
    <Application.Resources>
        <ResourceDictionary>
            <ResourceDictionary.MergedDictionaries>
                <ResourceDictionary Source="Resources/Styles/Colors.xaml" />
                <ResourceDictionary Source="Resources/Styles/
Styles.xaml" />
            </ResourceDictionary.MergedDictionaries>
        </ResourceDictionary>
    </Application.Resources>
</Application>

```

```

    <ResourceDictionary>
        <Color x:Key="CompletedColor"> #1C8859 </Color>
        <Color x:Key="ActiveColor"> #D3D3D3 </Color>
    </ResourceDictionary>
</ResourceDictionary>
</Application.Resources>
</Application>

```

ResourceDictionary can define a wide range of different objects. We only need the two colors that we want to access from **ValueConverter**. Notice that these can be accessed by the key given to them and from any other XAML file using a static resource binding.

ValueConverter itself is referenced as a static resource but from a local scope.

Using ValueConverter

We want to use our brand-new **StatusColorConverter** object in **MainView**. Unfortunately, we have to jump through some hoops to make this happen. We need to do three things:

- Define a namespace in XAML
- Define a local resource that represents an instance of the converter
- Declare that we want to use the converter in the binding

Let's start with the namespace:

1. Open **Views/MainView.xaml**.
2. Add the following namespace to the page:

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/
maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/
xaml"
              xmlns:local="clr-namespace:DoToo.ViewModels"
              xmlns:converters="clr-namespace:DoToo.Converters"
              x:Class="DoToo.Views.MainView" Title="Do Too!>

```

Add a **Resource** node to the **MainView.xaml** file:

1. Open **Views/MainView.xaml**.
2. Add the following **ResourceDictionary** element, shown in bold under the root element of the XAML file:

```

<ContentPage ...>
  <ContentPage.Resources>
    <ResourceDictionary>
      <converters:StatusColorConverter
x:Key="statusColorConverter"/>
    </ResourceDictionary>
  </ContentPage.Resources>
  <ContentPage.ToolbarItems>
    <ToolbarItem Text="Add" Command="{Binding AddItem}" />
  </ContentPage.ToolbarItems>
  <Grid ...>
  </Grid>
</ContentPage>

```

This has the same form as the global resource dictionary, but since this one is defined in **MainView**, it can only be accessed from there. We could have defined this in the global resource dictionary, but it's usually more efficient to define objects that you only consume in one place as close to that place as possible.

The last step is to add the converter:

1. Locate the **BoxView** node in the XAML file.
2. Add the **BackgroundColor** XAML, which is marked in bold:

```

<BoxView Grid.RowSpan="2"
  BackgroundColor="{Binding Item.Completed,
Converter={StaticResource statusColorConverter}}" />

```

What we have done here is bind a **bool** value to a property that takes a **Color** object. Right before the data binding takes place, however, **ValueConverter** converts the **bool** value into a color. This is just one of the many cases where **ValueConverter** comes in handy. Keep this in mind when you define the GUI.

Navigating to an item using a command

We want to be able to see the details for a selected to-do list item. When we tap a row, we should navigate to the item in that row.

To do this, we need to add the following code:

1. Open **ViewModels/MainViewModel.cs**.

2. Add the **SelectedItem** property, the **OnSelectedItemChanging** event handler, and the **NavigateToItemAsync** method to the class:

```
[ObservableProperty]
TodoItemViewModel selectedItem;
partial void OnSelectedItemChanging(TodoItemViewModel value)
{
    if (value == null)
    {
        return;
    }
    MainThread.BeginInvokeOnMainThread(async () => {
        await NavigateToItemAsync(value);
    });
}
private async Task NavigateToItemAsync(TodoItemViewModel item)
{
    var itemView = services.GetRequiredService<ItemView>();
    var vm = itemView.BindingContext as ItemViewModel;
    vm.Item = item.Item;
    itemView.Title = "Edit todo item";
    await Navigation.PushAsync(itemView);
}
```

The **SelectedItem** property is a property that we will data-bind to **ListView**. When we select a row in **ListView**, this property is set to the **TodoItemViewModel** object that represents that row. We are using the **ObservableProperty** attribute here to carry out its **PropertyChanged** magic. However, since the setter is being generated through the **ObservableProperty** attribute, there is no place to add additional code to the property. Luckily, the **ObservableProperty** source generator also adds two partial methods that can be implemented. We are using **OnSelectedItemChanging** to add additional functionality to the setter. The other partial method is **OnSelectedItemChanged**. **OnSelectedItemChanging** is called before the property value has changed and **OnSelectedItemChanged** is called after the value has changed. Remember that you can always view the generated source to learn more about how these attributes are extending your code.

The **OnSelectedItemChanging** method then calls **NavigateToItem**, which creates a new **ItemView** view using the .NET MAUI dependency injection container. At this point, we change the **Title** of the view from "Add todo item" to "Edit todo item". We extract **ViewModel** from the newly created **ItemView** view and assign the current **TodoItem** object that

TodoItemViewModel contains. Confused? Remember that **TodoItemViewModel** wraps a **TodoItem** object, and it is that item that we want to pass to **ItemView**.

We are not done yet. Now, we need to data-bind the new **SelectedItem** property to the right place in the view:

1. Open **Views/MainView.xaml**.
2. Locate **ListView** and add the attributes in bold:

```
<ListView x:Name="ItemsListView" Grid.Row="1" RowHeight="70"
          ItemsSource="{Binding Items}"
          SelectedItem="{Binding SelectedItem}">
```

The **SelectedItem** attribute binds the **SelectedItem** property's **ListView** view to the **ViewModel** property. When the selection of an item in **ListView** changes, the **ViewModel** property's **SelectedItem** property is called and we navigate to the new and exciting views.

The **x:Name** attribute is for naming **ListView** because we need to make a small and ugly hack to make this work. **ListView** stays selected after the navigation is done. When we navigate back, it cannot be selected again until we select another row. To mitigate this, we need to hook up to the **ItemSelected** event of **ListView** and reset the selected item directly on **ListView**. This is not recommended because we shouldn't have any logic in our views, but sometimes, we have no other choice:

1. Open **Views/MainView.xaml.cs**.
2. Add the following code in bold:

```
public MainView(MainViewModel viewmodel)
{
    InitializeComponent();
    viewmodel.Navigation = Navigation;
    BindingContext = viewmodel;
    ItemsListView.ItemSelected += (s, e) =>
        ItemsListView.SelectedItem = null;
}
```

We should now be able to navigate to an item in the list. Next, we will mark it as complete.

Marking an item as complete using a command

We need to add a functionality that allows us to toggle items between **complete** and **active**. It is possible to navigate to the detailed view of the to-do list item, but this is too much work for a user. Instead, we'll add a **ContextAction** item to **ListView**. In iOS, for example, this is accessed by swiping left on a row:

1. Open **ViewModel/ToDoItemViewModel.cs**.
2. Add a **using** statement for **CommunityToolkit.Mvvm.Input**.
3. Add a command to toggle the status of the item and a piece of text that describes the status:

```
[RelayCommand]
void ToggleCompleted()
{
    Item.Completed = !Item.Completed;
    ItemStatusChanged?.Invoke(this, new EventArgs());
}
```

Here, we have added a command for toggling the state of an item. When executed, it inverses the current state and raises the **ItemStatusChanged** event so that subscribers are notified. To change the text of the context action button depending on the status, we added a **StatusText** property. This is not recommended practice because we are adding code that only exists because of a specific UI case to **ViewModel1**. Ideally, this would be handled by the view, perhaps by using **ValueConverter**. To save us from having to implement these steps, however, we have left it as a string property:

1. Open **Views/MainView.xaml**.
2. Locate the **ListView.ItemTemplate** node and add the following **ViewCell.ContextActions** node:

```
<ListView.ItemTemplate>
  <DataTemplate>
    <ViewCell>
      <ViewCell.ContextActions>
        <MenuItem Text="{Binding StatusText}" Command="{Binding
ToggleCompletedCommand}" />
      </ViewCell.ContextActions>
      <Grid Padding="15,10">
        ...
```

```

        </Grid>
    </ViewCell>
</DataTemplate>
</ListView.ItemTemplate>

```

Creating the filter toggle function using a command

We want to be able to toggle between viewing active items only and all the items. We will create a simple mechanism to do this.

Hook up the changes in **MainViewModel** as follows:

1. Open **ViewModels/MainViewModel.cs** and locate **ItemStatusChangeMethod**.
2. Add the implementation to the **ItemStatusChanged** method and a property called **ShowAll** to control the filtering:

```

private void ItemStatusChanged(object sender, EventArgs e)
{
    if (sender is TodoItemViewModel item)
    {
        if (!ShowAll && item.Item.Completed)
        {
            Items.Remove(item);
        }
        Task.Run(async () => await repository.UpdateItemAsync(item.Item));
    }
}

[ObservableProperty]
bool showAll;

```

The **ItemStatusChanged** event handler is triggered when we use the context action from the previous section. Since the sender is always an object, we try to cast it to **TodoItemViewModel**. If this is successful, we check whether we can remove it from the list if **ShowAll** is not **true**. This is a small optimization; we could have called **LoadData** and reloaded the entire list, but since the **Items** list is set to **ObservableCollection**, it communicates to **ListView** that one item has been removed from the list. We also call the repository to update the item to persist the change of status.

The **ShowAll** property is what controls which state our filter is in. We need to adjust the **LoadData** method to reflect this:

1. Locate the **Load** method in **MainViewModel**.
2. Add the following lines of code marked in bold:

```
private async Task LoadDataAsync()
{
    var items = await repository.GetItemsAsync();
    if (!ShowAll)
    {
        items = items.Where(x => x.Completed == false).ToList();
    }
    var itemViewModels = items.Select(i =>
        CreateTodoItemViewModel(i));
    Items = new ObservableCollection<TodoItemViewModel> (itemViewModels);
}
```

If **ShowAll** is **false**, we limit the content of the list to the items that have not been completed. We can do this either by having two methods, **GetAllItems()** and **GetActiveItems()**, or by using a **filter** argument that can pass to **GetItemsAsync()**. Take a minute to think about how we could implement this.

Let's add the code that toggles the filter:

1. Open **ViewModels/MainViewModel.cs**.
2. Add the **FilterText** and **ToggleFilterAsync** properties:

```
[RelayCommand]
private async Task ToggleFilterAsync()
{
    ShowAll = !ShowAll;
    await LoadDataAsync();
}
```

The **ShowAll** property is a Boolean value, and that does not display well in a human-readable form. We will use another **ValueConverter** to change the status into a human-readable form:

1. Create a new class in the **Converters** folder named **FilterTextConverter.cs**.
2. Add the following code:

```
using System;
using System.Globalization;
internal class FilterTextConverter : IValueConverter
```

```
{
    public object Convert(object value, Type targetType, object
parameter, CultureInfo culture)
    {
        return (bool)value ? "All" : "Active";
    }
    public object ConvertBack(object value, Type targetType,
object parameter, CultureInfo culture)
    {
        return null;
    }
}
```

FilterTextConverter is very similar to the previous converter we created. The difference is that in the **Convert** method, we convert a **bool** value into the "All" or "Active" string. This converter will be used in the view to change the value of **ShowAll** into a value more suitable for display in the user interface.

The logic for the **ToggleFilterAsync** command is a simple inversion of the state and then a call to **LoadDataAsync**. This, in turn, causes the list to be reloaded.

Before we can filter the items, we need to hook up the filter button to **Command** and **Converter**:

1. Open **Views/MainView.xaml**.
2. Add the following highlighted entry to **ResourceDictionary**:

```
<ResourceDictionary>
    <converters:StatusColorConverter x:Key=
"statusColorConverter"/>
    <converters:FilterTextConverter
x:Key="filterTextConverter"/>
</ResourceDictionary>
```

3. Locate the button that controls the filter (the only button in the file).
4. Adjust your code to reflect the following code:

```
<Button Text="{Binding ShowAll,Converter={StaticResource
filterTextConverter}, StringFormat='Filter: {0}'}"
Command="{Binding ToggleFilterAsyncCommand}" />
```

We have now finished with this feature! However, our app isn't very attractive; we'll deal with this in the following section.

Laying out the contents

This last section is about making the app look a bit nicer. We will just scratch the surface of the possibilities here, but this should give you some ideas about how styling works.

Setting an application-wide background color

Styles are a great way of applying styling to elements. They can be applied either to all elements of a type or the elements referenced by a key if you add an **x:Key** attribute:

1. Open **App.xaml**.
2. Add the following XAML, which is in bold, to the file:

```
<ResourceDictionary>
  <Style TargetType="NavigationPage">
    <Setter Property="BarBackgroundColor" Value="#A25EBB" />
    <Setter Property="BarTextColor" Value="FFFFFF" />
  </Style>
  <Style x:Key="FilterButton" TargetType="Button">
    <Setter Property="Margin" Value="15" />
    <Setter Property="BorderWidth" Value="1" />
    <Setter Property="BorderColor" Value="Silver" />
    <Setter Property="TextColor" Value="Black" />
  </Style>
  <Color x:Key="CompletedColor">#1C8859</Color>
  <Color x:Key="ActiveColor">#D3D3D3</Color>
</ResourceDictionary>
```

The first style we will apply is a new background color and text color to the navigation bar. The second style will be applied to the filter button. We can define a style by setting **TargetType**, which tells .NET MAUI which type of object this style can be applied to. We can then add one or more properties that we want to set. The result will be the same as if we had added these properties directly to the element in the XAML code.

Styles that lack the **x:Key** attribute are applied to all instances of the type defined in **TargetType**. Styles that have a key must be explicitly assigned in the XAML of the user interface. We will see examples of this when we define the filter button in the next section.

Laying out the MainView and ListView items

In this section, we will improve the appearance of **MainView** and **ListView**. Open **Views/MainView.xaml** and apply the changes in bold in the XAML code in each of the following sections.

The filter button

The filter button allows us to toggle the state of the list to show only the active to-do items or all the to-do items. Let's style it to make it stand out a bit in the layout:

1. Find the filter button.
2. Make the following changes:

```
<Button Style="{DynamicResource FilterButton}"
        Text="{Binding ShowAll, Converter={StaticResource
filterTextConverter}, StringFormat='Filter: {0}'}"
        BackgroundColor="{DynamicResource ActiveColor}"
        TextColor="Black"
        Command="{Binding ToggleFilterCommand}">
    <Button.Triggers>
        <DataTrigger TargetType="Button" Binding="{Binding ShowAll}"
Value="True">
            <Setter Property="BackgroundColor" Value="{DynamicResource
CompletedColor}" />
            <Setter Property="TextColor" Value="White" />
        </DataTrigger>
    </Button.Triggers>
</Button>
```

The style is applied using **DynamicResource**. Anything defined in a resource dictionary, either in the **App.xaml** file or in the local XAML file, is accessible through it. Then, we set **BackgroundColor**, again setting **DynamicResource** to **ActiveColor** and **TextColor** to **Black**.

The **Button.Triggers** node is a useful feature. We can define several types of triggers that fire when certain criteria are met. In this case, we use a data trigger that checks whether the value of **ShowAll** changes to **true**. If it does, we set **TextColor** to white and **BackgroundColor** to **CompletedColor**. The coolest part is that when **ShowAll** becomes **false** again, it switches back to whichever value it was before.

Touching up ListView

ListView could use a couple of minor changes. The first change is formatting the due date string to a more human-readable format and the second is changing the color of the **Completed** label to a nice green tint:

1. Open **Views/MainView.xaml**.
2. Locate the labels that bind **Item.Due** and **Item.Completed** in **ListView**:

```
<Label Grid.Column="1" Grid.Row="1"
      Text="{Binding Item.Due, StringFormat='{0:MMMM d, yyyy}}'"
      FontSize="Micro" />
<Label Grid.Column="1" Grid.Row="1"
      HorizontalTextAlignment="End"
      Text="Completed"
      IsVisible="{Binding Item.Completed}"
      FontSize="Micro"
      TextColor="{StaticResource CompletedColor}" />
```

Here, we added a formatting string to the binding to format the date using a specific format. In this case, we used the **0:MMMM d, yyyy** format, which will display the date as a string in the format of, for example, May 5, 2020.

We also added a text color to the **Completed** label that is only visible if an item is completed. We did this by referencing our dictionary in **App.xaml**.

Now that all the code changes are complete, run our application. Here is a small gallery of screenshots that should match your application:

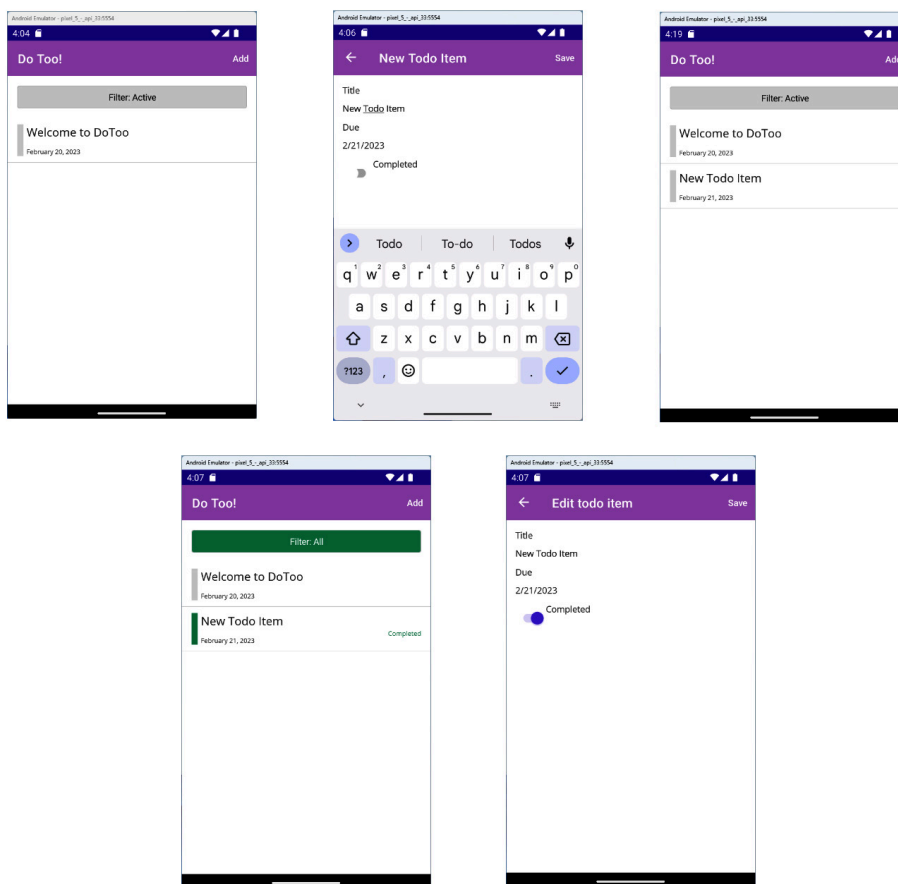


Figure 2.15 – DoToo on Android

Summary

You should now have a good grasp of all the steps involved in creating a .NET MAUI app from scratch. In this chapter, we learned about the project structure and the important files in a newly created project. We talked about dependency injection and learned the basics of MVVM by creating all the views and the **ViewModel** classes needed. We also covered data storage in SQLite to persist data on our device in a fast and secure way. Using the knowledge you've gained from this chapter, you should now be able to create the backbone of any app you'd like.

The next chapter will focus on upgrading an existing Xamarin.Forms application to .NET MAUI.